

Relatório Técnico: Implementação de Estruturas de Dados

1. Dados de Identificação

- **Aluno:** Adrian Henrique Olinek da Silva
- **Curso:** Análise e Desenvolvimento de Sistemas (ADS)
- **Disciplina:** Estrutura de Dados

2. Visão Geral do Projeto

O presente trabalho prático visa consolidar os conceitos fundamentais de estruturas de dados lineares através da implementação de uma Pilha (Stack) e uma Fila (Queue) na linguagem Java. O grande diferencial técnico desta entrega é o atendimento estrito ao **Desafio Adicional**: todas as estruturas foram construídas sobre vetores primitivos (Object[]), gerenciando os índices, memória e ponteiros de forma totalmente manual. Nenhuma classe utilitária de coleções ou métodos nativos de deleção e deslocamento (equivalentes ao array_pop ou array_shift) foram utilizados para a lógica central, garantindo rigor algorítmico.

3. Exercício 1: Pilha (Histórico de Navegação)

3.1. O que foi pedido

O professor solicitou o desenvolvimento de um programa capaz de simular o histórico de páginas acessadas por um usuário utilizando uma estrutura de Pilha. O sistema deveria operar rigidamente sob o conceito **LIFO (Last In, First Out)** - o último elemento a entrar deve ser o primeiro a sair. As operações exigidas foram: push (adicionar), pop (voltar/remover), peek (visualizar página atual), isEmpty (verificar se está vazia) e listar todas as páginas.

3.2. O que foi realizado e Como Funciona (Resolução do Desafio)

Para resolver este exercício de forma avançada, foi criada a classe genérica Pilha<T> que encapsula um vetor e um ponteiro de controle inteiro chamado topo (inicializado em -1). A estrutura funciona da seguinte maneira:

- **Adicionar página (Push):** O sistema verifica se o array atingiu a capacidade máxima. Se

sim, ele realiza o redimensionamento dinâmico. O ponteiro topo é então incrementado e a nova página é salva no novo índice.

- **Voltar página (Pop):** Aqui o desafio adicional foi cumprido. Para remover o elemento do histórico **sem usar métodos nativos**, o algoritmo acessa o elemento no índice topo, salva seu valor para retorno, substitui a posição original no vetor por null (evitando vazamento de memória) e decrementa o ponteiro topo. O elemento é removido logicamente sem a necessidade de realocar todo o array.
- **Visualizar atual (Peek):** Retorna rapidamente a página localizada no índice do topo atual sem realizar a remoção.
- **Listar:** Um iterador percorre do topo até o índice 0, imprimindo as páginas na ordem reversa, provando visualmente o conceito LIFO.

Demonstração da adição (push) e visualização (peek) no histórico]

```
Administrador: Prompt de Comando - java Main
2. Voltar para a página anterior
3. Visualizar página atual
4. Verificar se está vazio
5. Listar páginas
0. Voltar ao menu principal
Escolha uma opção: 1
Digite o endereço da página: corinthians.com.br
Página adicionada ao histórico.

=== HISTÓRICO DE NAVEGAÇÃO ===
1. Adicionar página
2. Voltar para a página anterior
3. Visualizar página atual
4. Verificar se está vazio
5. Listar páginas
0. Voltar ao menu principal
Escolha uma opção: 3
Página atual: corinthians.com.br
```

Demonstração de listagem das páginas do histórico em ordem LIFO

```
=== HISTÓRICO DE NAVEGAÇÃO ===
1. Adicionar página
2. Voltar para a página anterior
3. Visualizar página atual
4. Verificar se está vazio
5. Listar páginas
0. Voltar ao menu principal
Escolha uma opção: 5
Histórico de páginas:
1. corinthians.com.br
2. corinthians.com.br
3. 0
```

4. Exercício 2: Fila (Atendimento de Clientes)

4.1. O que foi pedido

A segunda etapa exigia a simulação de uma fila de atendimento ao cliente, baseada no conceito **FIFO (First In, First Out)** - o primeiro a entrar é o primeiro a sair. As regras operacionais eram implementar os métodos: enqueue (adicionar cliente), dequeue (atender cliente), front (visualizar próximo), isEmpty e listar todos os clientes.

4.2. O que foi realizado e Como Funciona (Resolução do Desafio)

Foi implementada a classe `Fila<T>` adotando a arquitetura altamente otimizada de **Fila Circular**. Esta abordagem foi a chave para resolver o desafio extra de não utilizar deslocamento de array (o que ocorreria ao usar um `shift`). Em vez de empurrar todos os itens para a esquerda a cada atendimento, o sistema trabalha com dois ponteiros independentes: início e fim.

- **Adicionar Cliente (Enqueue):** O cliente é adicionado no índice apontado por fim. Para manter a fila infinita dentro de um espaço finito de array, o novo valor de fim é calculado com o operador matemático de módulo (resto da divisão): $(\text{fim} + 1) \% \text{tamanho_do_array}$.
- **Atender Cliente (Dequeue):** Recupera o cliente apontado por início. Para remover de fato sem usar recursos do Java, a posição atual no vetor recebe null, e o ponteiro início avança um passo à frente de forma circular. O algoritmo atende o primeiro cliente sem afetar a posição de nenhum outro na memória.
- **Visualizar Próximo (Front):** Lê instantaneamente o valor presente no ponteiro início.
- **Listar:** O laço percorre a fila partindo do início, controlando rigorosamente quantas iterações ocorrem com base na quantidade total de clientes no momento, mostrando a fila na sequência correta de atendimento.

Demonstração de inserção de clientes]

```

Prompt de Comando - java Main
C:\Users\Adrian-PC\Desktop\projeto modulo 4\estrutura de dados atividade>java Main

=== ESTRUTURAS DE DADOS ===
1. Histórico de navegação - Pilha
2. Atendimento de clientes - Fila
0. Sair
Escolha uma opção: 2

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia
5. Listar clientes
0. Voltar ao menu principal
Escolha uma opção: 1
Digite o nome do cliente: Burnes
Cliente adicionado à fila.

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia
5. Listar clientes
0. Voltar ao menu principal
Escolha uma opção: 1
Digite o nome do cliente: João
Cliente adicionado à fila.
```

Demonstração do atendimento ordenado (dequeue) dos clientes]

```

Prompt de Comando - java Main

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia
5. Listar clientes
0. Voltar ao menu principal
Escolha uma opção: 2
Atendendo: Burnes

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia
5. Listar clientes
0. Voltar ao menu principal
Escolha uma opção: 2
Atendendo: J?o

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia
5. Listar clientes
0. Voltar ao menu principal
Escolha uma opção: 3
Não há clientes aguardando atendimento.
```

Demonstração da verificação e alertas de fila vazia

```

Prompt de Comando - java Main

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia
5. Listar clientes
0. Voltar ao menu principal
Escolha uma opção: 4
A fila está vazia.

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia
5. Listar clientes
0. Voltar ao menu principal
Escolha uma opção: 5
A fila está vazia.

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia
5. Listar clientes
0. Voltar ao menu principal
Escolha uma opção: 1
Digite o nome do cliente: vinicius
```

Demonstração da listagem final da fila]

```

Prompt de Comando - java Main
Cliente adicionado à fila.

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia
5. Listar clientes
0. Voltar ao menu principal
Escolha uma opção: 1
Digite o nome do cliente: paulo
Cliente adicionado à fila.

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia
5. Listar clientes
0. Voltar ao menu principal
Escolha uma opção: 5
Fila de atendimento:
1. vinicius
2. paulo

=== ATENDIMENTO DE CLIENTES ===
1. Adicionar cliente
2. Atender próximo cliente
3. Visualizar próximo cliente
4. Verificar se está vazia

```

5. Avaliação Técnica de Complexidade de Algoritmos (Big-O)

Graças à implementação manual via manipulação matemática de índices exigida pelo desafio, o projeto garantiu um altíssimo desempenho. A tabela abaixo resume a eficácia computacional alcançada pelo trabalho:

Estrutura	Operações Principais	Tempo de Execução	Justificativa Arquitetural
Pilha LIFO	Push / Pop	$O(1)$ - Constante	Os itens são gravados e removidos acessando diretamente o espaço de memória ditado pelo topo, sem a necessidade

Estrutura	Operações Principais	Tempo de Execução	Justificativa Arquitetural
			de reordenar o vetor.
Fila FIFO	Enqueue / Dequeue	$O(1)$ - Constante	Com o uso da Fila Circular, o ponteiro de início apenas avança. Nenhuma reatribuição $O(n)$ de <i>shift</i> foi realizada no array.

6. Conclusão Final

As duas estruturas propostas cumpriram não apenas suas funções simulatórias do cotidiano (histórico de navegação e balcão de atendimento), como demonstraram o funcionamento prático da alocação primitiva de memória. A abstenção do uso de métodos facilitadores nativos do Java permitiu comprovar o total entendimento de LIFO, FIFO e algoritmos de gerenciamento dinâmico sem vazamento de memória (Memory Leaks).